Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 9

Disciplina: Tehnici de programare

Tema: ” Funcții și excepții ”

A efectuat : st. gr.SI – 264 Șoimaru Gabriel

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Lucrarea A

A1. Scheletul și funcția care întoarce o fișă

Un rând de eșec arată așa: Failed password for root from 203.0.113.9 port 51230 ssh2. Scrie o funcție esec(linie) care întoarce (user, ip) dacă rândul e un eșec, altfel None.

A2. Lista de eșecuri, robustă la rânduri stricate

citeste_esecuri(cale) parcurge fișierul și strânge fișele (user, ip) într-o listă de dicționare. Sare peste rândurile care nu sunt eșecuri. Pune citirea fișierului într-un try/except, ca un fișier lipsă să dea un mesaj clar, nu un traceback.

A3. Numărarea pe IP: o funcție cu return

Numeri de câte ori apare fiecare adresă. Poți cu un dicionar și .get, sau cu Counter din collections, care face exact asta. Funcția ÎNTOARCE numărătoarea, nu o tipărește.

A4. Rezumatul

O funcție main() leagă totul: citește, numără, tipărește rezumatul. Rulezi din rădăcina depozitului.

Lucrarea B

B1. suspecti() cu valoare implicită și validare

O funcție suspecti(counts, prag=10) întoarce adresele cu cel puțin prag eșecuri, sortate descrescător. Pragul are o valoare implicită, dar îl poți schimba. Și un prag absurd (sub 1) nu are sens: îl oprești cu raise.

B2. Validarea, verificată

Verifică faptul că un prag absurd e oprit cu un mesaj clar:

B3. argparse și raportul JSON

main() primește jurnalul, un --prag (implicit 10, type=int) și un --json. Leagă funcțiile din parse_auth cu suspecti și scoate fie un rezumat citibil, fie un raport JSON.

B4. Un test care prinde regresia

Scrie arsenal/aparare/test_detector.py cu două teste cu date scrise de mână: numărarea e corectă și pragul taie

unde trebuie. Rulează fără jurnal real.

EFECTUAREA LUCRĂRII

Lucrarea A

A1

```python
import re
TIPAR = re.compile(r"Failed password for (?:invalid user )?(\S+) from (\S+) port")
def esec(linie):
    m = TIPAR.search(linie)
    if m is None:
        return None
    else:
        return (m.group(1), m.group(2))

print(esec("... Failed password for root from 203.0.113.9 port 51230 ssh2"))
print(esec("<linie stricata fara structura>"))
```

Figura 1.1.1 – Codul A1

```
('root', '203.0.113.9')
None
```

Figura 1.1.2 – Rezultatul A1

A2

```python
def citeste_esecuri(cale):
    esecuri = []
    try:
        f = open(cale, encoding="utf-8")
    except FileNotFoundError:
        print("nu gasesc", cale)
        return esecuri
    with f:
        for linie in f:
            r = esec(linie)
            if r:
                esecuri.append({"user": r[0], "ip": r[1]})
    return esecuri

esecuri = citeste_esecuri("arsenal/aparare/start/auth.log")
print("esecuri:", len(esecuri))
```

Figura 1.2.1 – Codul A2

```
esecuri: 15
```

Figura 1.2.2 – Rezultatul A2

A3

```python
from collections import Counter

def pe_ip(esecuri):
    cnt = (Counter(e["ip"] for e in esecuri))
    return cnt

counts = pe_ip(esecuri)
for ip, n in counts.most_common():
    print(f" {ip}: {n}")
```

Figura 1.3.1 – Codul A3

```
203.0.113.9: 13
198.51.100.7: 2
```

Figura 1.3.2 – Rezultatul A3

A4

```python
import argparse
def main():
    p = argparse.ArgumentParser()
    p.add_argument("cale")
    a = p.parse_args()
    esecuri = citeste_esecuri(a.cale)
    counts = pe_ip(esecuri)
    print(f"Esecuri de autentificare: {len(esecuri)}")
    for ip, n in counts.most_common():
        print(f" {ip}: {n}")

if __name__ == "__main__":
    main()
```

Figura 1.4.1 – Codul A4

```
PS D:\Desktop\Gabi\Uni\Tehnici de programare\LabTP9> python -m arsenal.aparare.parse_auth arsenal/aparare/start/auth.log
Esecuri de autentificare: 15
 203.0.113.9: 13
 198.51.100.7: 2
```

Figura 1.4.2 – Rezultatul A4

Lucrarea B

B1

```python
def suspecti(counts, prag=10):
    if prag < 1:
        raise ValueError(f"prag invalid: {prag} (cel putin 1)")
    out = []
    for ip, n in counts.items():
        if n >= prag:
            out.append((ip, n))
    return sorted(out, key = lambda x: x[1], reverse=True)

from collections import Counter
c = Counter({"203.0.113.9": 13, "198.51.100.7": 2})
print(suspecti(c))
print(suspecti(c, prag=2))
```

Figura 2.1.1 – Codul B1

```
[('203.0.113.9', 13)]
[('203.0.113.9', 13), ('198.51.100.7', 2)]
```

Figura 2.1.2 – Rezultatul B1

B2

```
PS D:\Desktop\Gabi\Uni\Tehnici de programare\LabTP9> python -c "from arsenal.aparare.detector import suspecti; suspecti({}, 0)"
[('203.0.113.9', 13)]
[('203.0.113.9', 13), ('198.51.100.7', 2)]
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    from arsenal.aparare.detector import suspecti; suspecti({}, 0)
                                                    ^^^^^^^^^^^^^^
  File "D:\Desktop\Gabi\Uni\Tehnici de programare\LabTP9\arsenal\aparare\detector.py", line 4, in suspecti
    raise ValueError(f"prag invalid: {prag} (cel putin 1)")
ValueError: prag invalid: 0 (cel putin 1)
```

Figura 2.2 – Rezultatul testului B3

B3

```python
#B3
import argparse, json
from collections import Counter
from arsenal.aparare.parse_auth import citeste_esecuri
def main():
    p = argparse.ArgumentParser()
    p.add_argument("log")
    p.add_argument("--prag", type=int, default=10)
    p.add_argument("--json", action="store_true")
    a = p.parse_args()
    esecuri = citeste_esecuri(a.log)
    counts = Counter(e["ip"] for e in esecuri)
    sus = suspecti(counts, a.prag)
    out = {
        "esecuri": esecuri,
        "pe_ip": counts,
        "suspecti": sus
        }
    if a.json:
        print(json.dumps(out, indent=2)); return
    else:
        print(f"Esecuri de autentificare: {len(esecuri)}")
        for ip, n in counts.most_common():
            print(f"  {ip}: {n}")
        print(f"Suspecti (prag {a.prag}):")
        for ip, n in sus:
            print(f"  {ip} ({n} incercari) BRUTE FORCE")

if __name__ == "__main__":
    main()
```

Figura 2.3.1 – Codul B3

```
PS D:\Desktop\Gabi\Uni\Tehnici de programare\LabTP9> python -m arsenal.aparare.detector arsenal/aparare/start/auth.log
Esecuri de autentificare: 15
  203.0.113.9: 13
  198.51.100.7: 2
Suspecti (prag 10):
  203.0.113.9 (13 incercari) BRUTE FORCE
```

Figura 2.3.2 – Rezultatul B3

B4

```python
from collections import Counter
from arsenal.aparare.detector import suspecti

def test_pragul_taie():
    c = Counter({"a": 13, "b": 2})
    assert suspecti(c, 10) == [("a", 13)]

def test_prag_invalid():
    import pytest
    with pytest.raises(ValueError):
        suspecti({}, 0)
```

Figura 2.4.1 – Codul testului B4

```
PS D:\Desktop\Gabi\Uni\Tehnici de programare\LabTP9> pytest -q arsenal/aparare/test_detector.py
..
2 passed in 0.03s
```

Figura 2.4.2 – Rezultatul testului B4

CONCLUZII

În această lucrare am folosit funcții în crearea fișierului parse_auth.py pentru a găsi eșecurile din fișierul auth.log și am creat un detector de brute force cu prag reglabil care salvează rezultatele într-un fișier JSON.